

Vector-Guided Embezzler: A Reconstruction Attack Against Retrieval-Augmented Generation in Agentic AI Communication

Lishuang Hu, Hao Wang[✉], Tao Xiang[✉], *Senior Member, IEEE*, Zhuo Chen[✉], *Member, IEEE*, Jialing He[✉], *Member, IEEE*, and Shangwei Guo[✉], *Member, IEEE*

Abstract—Agentic AI systems increasingly rely on Retrieval-Augmented Generation (RAG) to ground autonomous decision-making in external knowledge. They become critically dependent on distributed, often third-party, infrastructure — introducing new vulnerabilities at the intersection of embodied intelligence and networked communication. While prior work has examined privacy risks from end-users, we identify and formalize a more systemic threat: the LLM API provider itself acting as a malicious adversary within the agentic ecosystem. In this paper, we introduce a new threat model where the LLM API provider is the malicious adversary and demonstrate this vulnerability. Specifically, we propose a novel three-stage attack, Embezzler, to reconstruct the private knowledge base of a RAG system. The proposed Embezzler is to first infer the embedding model of the RAG system and use this surrogate model to probe new, non-redundant text chunks within the embedding space, rather than the semantic space. Then, we systematically explore and refine the embedding space to maximize the amount of unique chunks reconstructed with minimal adversarial queries. We theoretically analyze the effectiveness of our Embezzler and conduct extensive experiments under various system settings. Our results show that our Embezzler consistently and significantly outperforms four baseline methods and reconstructs nearly 90% of the RAG knowledge base. A component-wise ablation study further validates the critical role of each stage. Our findings highlight the severe privacy risks of deploying RAG systems with third-party LLM APIs in Agentic AI communication.

Index Terms—Retrieval-augmented generation, reconstruction attack, LLM, communication.

I. INTRODUCTION

LARGE Language Models (LLMs) [1], [2], [3], [4] serve as cognitive engines for autonomous agents in next-generation communication and networking systems, enabling real-time reasoning, adaptive control, and collaborative intelligence in decentralized environments. However,

their susceptibility to hallucination, i.e., generating fluent but factually incorrect outputs, poses critical risks when deployed in safety-sensitive, embodied agent ecosystems such as edge networks, robotic swarms, or distributed IoT infrastructures. To mitigate this, Retrieval-Augmented Generation (RAG) has emerged as a foundational architecture [5], [6], [7], [8], [9], allowing Agentic AI systems to dynamically ground their decisions in external, verifiable knowledge bases. By retrieving contextually relevant text chunks prior to response generation, RAG enhances factual integrity, supports continual learning from evolving environments, and enables embodied agents to operate reliably under uncertainty [10], [11], [12], [13]. This capability is indispensable in networked agent systems where accurate, real-time knowledge retrieval underpins mission-critical coordination, resource allocation, and security-aware adaptation.

A RAG system usually follows a two-step workflow: (1) retrieving relevant text chunks from an external knowledge base by comparing the query and the chunks in the vector space of an embedding model, and (2) generating a coherent response by passing the query and the retrieved chunks to the LLM. Modern LLMs necessitate deployment on GPU/TPU clusters to sustain real-time inference. The self-hosted clusters would incur exorbitant operational expenses and is impractical for most organizations and companies [14], [15], [16]. To mitigate such computational cost, resource-constrained RAG service providers often deploy RAG systems via third-party LLM APIs through pay-per-query billing models [17]. However, reliance on untrustworthy providers introduces severe privacy risks: LLM API providers may log sensitive chunks (e.g., medical records, proprietary documents) that are transmitted as part of a query during communication. More seriously, malicious LLM API providers could further actively act as an authorized end user and construct a series of adversarial queries to recover or reconstruct the entire RAG knowledge bases.

Existing works [18], [19], [20], [21], [22], [23] do explore the privacy vulnerability of RAG systems and propose membership inference and data reconstructing attacks. These attacks assume that the end users are malicious and optimize malicious queries to lead RAG systems to output private information. But they overlook adversarial scenarios where LLM API providers themselves act as malicious attackers.

Cryptographic solutions [24], [25], [26], [27] like secure multi-party computation can theoretically mitigate risks from

Received 12 December 2025; revised 29 March 2026; accepted 10 May 2026. Date of publication 19 May 2026; date of current version 26 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 624B2024 and 62572079, and in part by the Postdoctoral Fellowship Program of CPSF under Grant GZB20250950. The associate editor coordinating the review of this article and approving it for publication was R. Zhang. (*Corresponding author: Zhuo Chen.*)

Lishuang Hu, Hao Wang, Tao Xiang, Jialing He, and Shangwei Guo are with the College of Computer Science, Chongqing University, Chongqing 401331, China (e-mail: hulishuang@stu.cqu.edu.cn; hwang@cqu.edu.cn; txiang@cqu.edu.cn; hejialing@cqu.edu.cn; swguo@cqu.edu.cn).

Zhuo Chen is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: chen.zhuo@bit.edu.cn).

Digital Object Identifier 10.1109/TCCN.2026.3694035

malicious LLM API providers by operating on ciphertexts. However, these solutions require substantial computational resources or communication costs and thus most RAG service providers upload unencrypted information to LLM API providers to obtain LLM services. Therefore, investigating the privacy of RAG systems under malicious LLM API providers is of great importance.

In this paper, we systematically evaluate the privacy risks of RAG systems by analyzing how adversarial LLM API providers could infer and reconstruct sensitive database content. We focus on two key questions: (1) *How can one infer the distribution of the knowledge base when the RAG service provider operates as a closed-box?* and (2) *How can an adversary reconstruct the maximum possible database content with minimal queries?*

Two key challenges emerge when reconstructing sensitive data from the private knowledge base of the RAG system. First, semantic similarity between text chunks does not always match their proximity in the *vector or embedding space* of the RAG system. Many existing attacks [21], [22] generate or filter queries based on semantic relevance. However, retrieval is determined solely by vector similarity (e.g., cosine similarity) after embedding. This gap becomes a significant issue when an embedding model maps related chunks to distant vectors or, conversely, maps unrelated chunks to nearby ones. The result is a mismatch between intuitive query strategies and actual retrieval outcomes. Second, covering the knowledge base broadly with minimal queries requires a balance between efficiency and completeness. Existing works mainly focus on optimizing a malicious prompt to trick the LLM into revealing the chunks that the RAG system has retrieved. But our goal is to generate adversarial queries that elicit more chunks that was not previously obtained.

Method. To address these challenges, we propose a novel three-stage attack, Embezzler, to reconstruct the private RAG knowledge base by inferring and exploring the embedding space. The core idea is to infer the embedding model and then use it as a surrogate model to probe the embedding space via minimizing adversarial queries and maximizing retrieval chunks that was not previously collected. We also theoretically analyze the effectiveness of our idea. The first stage is **embedding model inference**. We collect a limited sample of queries and their corresponding retrieval results from the RAG service provider. We also construct a model zoo with various candidate embedding models. By comparing these retrieved results under the candidate embedding models, we can theoretically determine a surrogate embedding model that produces retrieval results similar to those of the victim model. This surrogate embedding model is used for all future comparisons in the embedding space, rather than the semantic space.

To improve the effectiveness and completeness of the reconstruction attack, we leverage the surrogate embedding model to guide query generation and selection. Such adversarial queries explore and refine the embedding space and then maximally retrieves new and non-redundant chunks. In the **exploration** phase, we build a coarse-grained taxonomy of the knowledge base and formalize the query selection process as

an adaptive farthest-point sampling problem in the embedding space. We first generate a diverse pool of candidate queries using the observed queries as seeds. With the surrogate model, we iteratively select queries from this candidate pool by choosing the one that maximizes the minimum distance to the already-selected queries. In the **refinement** phase, we utilize the surrogate model to construct adversarial queries near the coarse-grained exploration queries and achieve refinement, i.e., fine-grained exploration, of the embedding space. We generate new candidate queries by strategically approximating the semantic meaning of already-retrieved chunks. Then, we use the surrogate model to define the divergence score of these candidate queries, and identify the refinement queries that can retrieve new and non-redundant chunks.

Evaluation. We conduct a comprehensive series of experiments under various system configurations to evaluate the performance of our Embezzler. Specifically, we evaluate the performance of our proposed method, Embezzler, against four baseline methods on three distinct datasets: MS MARCO, Natural Questions, and ChatDoctor. Our comprehensive evaluation demonstrates that Embezzler consistently achieved superior results across all datasets and metrics. Our Embezzler can recover nearly 90% of the RAG databases. Additionally, we conduct a component-wise ablation study, which confirmed that each stage of our Embezzler is crucial for maximizing attack efficiency and the total amount of recovered data.

Our main contributions are as follows:

- We introduce a novel threat model that focuses on malicious LLM API providers as adversaries, a scenario largely overlooked by previous work.
- We propose Embezzler, a novel reconstruction attack to recover the private RAG knowledge base, and we theoretically analyze its effectiveness.
- Our Embezzler can maximally reconstruct new chunks with minimal adversarial queries by exploring and refining the embedding space.
- Our comprehensive experiments demonstrate that our Embezzler consistently outperforms SOTA baselines and achieves a nearly 90% reconstruction rate.

The rest of this paper is organized as follows. Section II provides the necessary background on RAG and introduces privacy risks in RAG systems, followed by Section III which defines the adversarial objectives. Section IV details the technical implementation of our vector-guided reconstruction attack, Embezzler. The experimental setup and results are presented in Section V. Finally, Section VI concludes the paper and provides future research directions.

II. BACKGROUND AND RELATED WORK

A. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) systems have become a critical architectural component in Agentic AI deployments, where agents make real-time, context-aware decisions grounded in external knowledge [28], [29]. Initially, LLMs rely solely on their internal parameters and often struggles with domain-specific knowledge and prone to hallucinations. RAG first queries an external knowledge

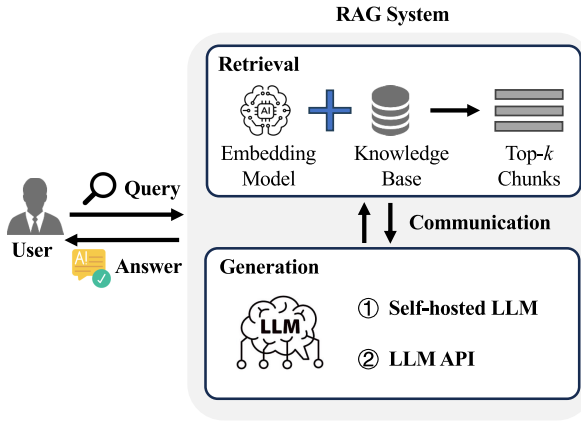


Fig. 1. A typical RAG system with two core components: the retrieval and generation modules.

base to retrieve relevant text chunks. This process substantially improves answer precision and mitigates model hallucinations by grounding responses in verifiable external data.

As shown in Fig 1, A typical RAG system consists of two core components: the retrieval and generation modules. The retrieval module (i.e., retriever with a text embedding model) identifies the most relevant information from the knowledge base in response to a query. The generation module receives the query and the corresponding chunks, and then generates a single, coherent response with a LLM.

Formally, a RAG service provider owns an original knowledge dataset and splits all text into small text chunks, denoted as $C = \{c_1, c_2, \dots, c_n\}$. It then uses an embedding model $E: \mathcal{T} \rightarrow \mathbb{R}^d$ to transform each chunk c_i into a vector c_i ,

$$c_i = E(c_i), \text{ where } i \in \{1, 2, \dots, n\} \text{ and } c_i \in \mathbb{R}^d. \quad (1)$$

We denote the set of all vectors as $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$. The service provider stores both the text chunks C and their corresponding vectors $\mathcal{C}$ for retrieval.

Retrieval. When received a query q , the retrieval module first transform q into a vector $\mathbf{q}$ using the embedding model, $\mathbf{q} = E(q) \in \mathbb{R}^d$. The retrieval module then calculates the cosine similarity between the vector of query and the vectors of text chunks in the knowledge base, i.e.,

$$s_i = d(\mathbf{q}, c_i) = \frac{\mathbf{q} \cdot c_i}{\|\mathbf{q}\| \|c_i\|}. \quad (2)$$

The retrieval module returns the chunks with top- k highest similarities

$$R = \text{Ret}(q), \quad (3)$$

where Ret is the retrieval algorithm and R is the set of the k most similar chunks in C .

Generation. The generation module mainly relay on an LLM that can be deployed by the RAG service provider itself or a cloud-based LLM API service [30]. For resource-constrained scenario, the RAG service provider can integrate third-party LLM APIs¹ to process queries through globally distributed endpoints, achieving elastic scalability without infrastructure overhead. After completing the generation process, the service provider combines the retrieval result R with

the query q and then sends $\{q, R\}$ to the LLM. After the LLM completes reasoning, it returns the output result to the RAG service provider, who in turn forwards the result to the user.

B. Privacy Risks in RAG Systems

Retrieval-Augmented Generation (RAG) systems, while enhancing the accuracy and relevance of LLM outputs, introduce critical privacy vulnerabilities. These risks manifest prominently in two categories: membership inference attacks and data reconstruction attacks.

Membership Inference Attacks. Membership Inference Attacks (MIAs) against RAG systems aim to determine whether a specific document resides in the knowledge base of a RAG system by analyzing its generative outputs. Existing methods exploit semantic dependencies between user queries and generated responses and leverages adversarial prompting or retrieval-distinctive patterns to infer membership. For example, Liu et al. [18] propose a mask-based MIA by evaluating prediction accuracy of masked target text chunks retrieved by RAG. Li et al. [19] introduce S2MIA, using semantic similarity between generated answers and target documents to infer membership with high AUC scores. Anderson et al. [31] design threshold-based prompts to amplify retrieval dependency, achieving precise inference in closed-box settings. Naseh et al. [32] develop stealthy interrogation attacks that use natural-language queries answerable only if the target chunk is retrieved, evading detection while requiring minimal queries. These MIAs expose the vulnerability of RAG systems to low-cost, high-confidence membership inference, even with limited adversarial access.

Data Reconstruction Attacks. Data Reconstruction Attacks in RAG systems aim to extract verbatim or semantically equivalent content from the private knowledge bases. These attacks [20], [21], [22], [23], [33], [34] primarily rely on crafting adversarial prompts to trigger unintended retrieval patterns or manipulate the LLM output, and then reconstruct the target data. For instance, Zeng et al. [20] construct query sequences designed to retrieve overlapping document chunks, which enables adversaries to stitch together leaked content. Di et al. [21] generate stealthy, context-aware queries using open-source LLMs to systematically probe the knowledge base. Jiang et al. [22] iteratively refine adversarial queries through agent-based automation to gradually expose private chunks. Qi et al. [33] inject cryptic instructions into prompts to hijack the generation logic of the RAG system and coerce verbatim data reconstruction. Cohen et al. [35] design self-replicating prompts that trigger recursive retrieval and propagate poisoned contexts. Peng et al. [34] embed hidden triggers in queries to activate backdoors in compromised LLMs, forcing leakage of stored. Wang et al. [23] craft seemingly benign prompts targeting LLM agent memory modules to extract private interaction logs.

Limitations. Current approaches, including MIAs and data reconstruction attacks, overlook critical risks posed by untrusted third-party LLM API providers. If such providers are not fully trustworthy, they could passively collect or actively exploit sensitive chunks transmitted by RAG service providers during their API interactions. Furthermore, existing

¹<https://github.com/cheahjhs/free-llm-api-resources>

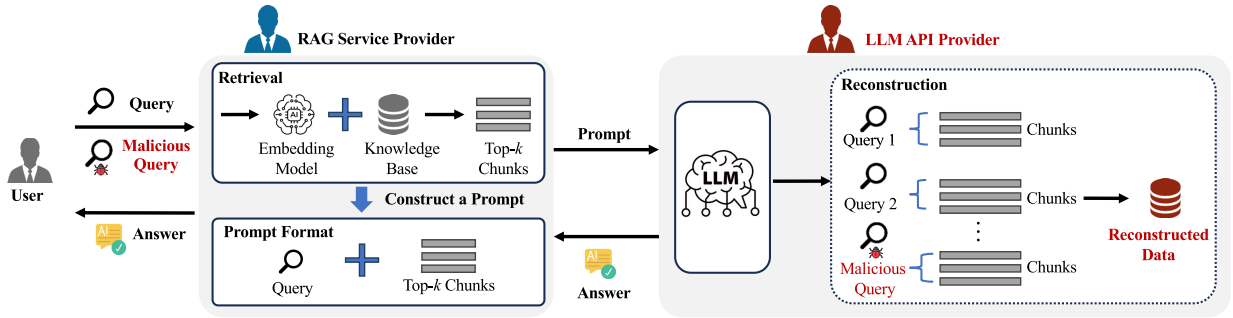


Fig. 2. Threat model. The untrustworthy LLM API provider is the adversary. The attacker aims to passively and actively reconstruct the knowledge base of the RAG service provider by balancing two objectives: maximizing data completeness while minimizing query overhead.

methods largely concentrate on the LLM component, using adversarial prompts to force LLMs to regurgitate retrieved chunks rather than synthesizing new content. However, they lack systematic exploration of attacks specifically designed to exploit vulnerabilities in the retriever.

III. OVERVIEW

A. Threat Model

We consider RAG systems deployed in agentic communication environments that rely on third-party LLM APIs as generative backends. The system primarily involves three participating parties: User, RAG Service Provider, and LLM API Provider. As shown in Fig. 2, the user submits a query to the RAG service provider and ultimately receives a synthesized response. Upon receiving the User's query, the RAG service provider retrieves relevant text chunks from its private knowledge base, forwards both the query and retrieved chunks to the LLM API provider for reasoning, and then relays the generated response back to the user. The LLM API provider receives the query and retrieved chunks from the RAG service provider, performs inference (e.g., answer generation, summarization), and returns the generated results to the RAG service provider.

Adversary Profile. In the RAG system, we assume the LLM API provider is untrustworthy. The adversarial posture stems fundamentally from profit-maximizing incentives inherent to commercial LLM operations. From an economic perspective, LLM API providers benefit from a “data flywheel” [36] where valuable vertical-domain knowledge (e.g., healthcare/finance chunks transmitted via APIs) directly enhances model competitiveness. From an efficiency perspective, both end users and RAG service providers cannot adopt ciphertext-based secure inference techniques, as most current implementations [24], [25], [26], [27] impose excessive computational overhead or necessitate costly interactive protocols.

Instead of passively collecting the data from the RAG service provider, we consider an active attack scenario where the LLM API provider itself implements proactive probing strategies. Through masquerading as authenticated users, the malicious LLM API provider reconstructs the knowledge base by actively probing the RAG service provider via carefully designed queries. Unlike existing research that focuses on malicious end-users restricted to querying via public interfaces and performing indirect inference via LLM outputs, the LLM API provider as an adversary possesses significantly enhanced

data observability and operational capabilities. Regarding observability, the provider occupies a central communication node, allowing for the direct capture of raw retrieved text chunks (context) and bypassing any semantic-level filters. In terms of capabilities, the provider can execute a passive-active reconnaissance strategy: silently accumulating data from legitimate user traffic while simultaneously injecting optimized queries to fill knowledge gaps.

Adversarial Capabilities. Following the adversary profile, we assume that the LLM API provider possesses both passive and active data collection capabilities. Passively, the adversary monitors and records pairs of user-submitted natural language queries and the corresponding retrieved chunks returned by the RAG service provider. Then, the adversary establishes an observational dataset during the API interactions. Actively, the adversary generates and injects carefully crafted probing queries into the system to elicit additional retrieval chunks and augments the reconstructed dataset. In our threat model, the adversary operates under a closed-box constraint, i.e., they cannot directly access the internal components of the RAG service provider, including the private knowledge base, the architecture of the embedding model, or the implementation details.

Adversarial Objectives. The adversary aims to comprehensively reconstruct the knowledge base of the RAG service provider, while balancing the twin objectives of

- **Completeness:** Ensuring the reconstructed knowledge base captures a comprehensive and representative portion of the original knowledge base;
- **Effectiveness:** Minimizing the number of queries needed to achieve a successful and complete reconstruction.

B. Our Approach

To achieve the objectives, one can maximize retrieved chunks by crafting queries with diverse semantic meanings. For example, an adversary might iteratively submit queries spanning varied topics or phrasing styles to trigger retrievals across distinct regions of the knowledge base. One can also leverage existing data reconstruction attacks [20], [21], [22] against RAG systems to extract private chunks since they also involve carefully designed strategies to maximally reconstruct the knowledge base. However, these approaches face a fundamental limitation: the discrepancy between the embedding space (where retrieval operates) and the seman-

tic space (perceived by humans). Queries designed to be semantically distinct may cluster closely in the embedding space. Therefore, naively maximizing semantic diversity in queries fails to guarantee coverage of the knowledge base, leading to redundant retrievals and suboptimal efficiency.

We propose a novel vector-guided probing strategy for exploring the embedding space of the RAG service provider to achieve a more comprehensive coverage of its original knowledge base. Our methodology is twofold. We begin by inferring a surrogate embedding model from passively observed RAG interactions. We then use this model to generate and select adversarial queries. These queries systematically probe the embedding space, balancing efficiency with the completeness of our reconstruction.

Theoretical Analysis. We now theoretically analyze the effectiveness of our approach. Its effectiveness stems from the principle that a query is more likely to retrieve new chunks as its distance from existing queries increases in the embedding space. This greater isolation reduces overlap with previously sampled probability mass.

Let $Q_{\text{obs}} = \{q_1, q_2, \dots, q_t\}$ be the embeddings of existing queries. For any new query q , its minimum distance to existing queries is defined as the distance to the nearest neighbor in Q_{obs} :

$$d(E(q), E(Q_{\text{obs}})) = \min_{q_i \in Q_{\text{obs}}} d(E(q), E(q_i)). \quad (4)$$

Assume each query retrieves data chunks within a fixed radius $\epsilon > 0$ from its embedding. The effective retrieval region for query q consists of chunks in its ϵ -ball that haven't been covered by existing queries

$$C_{\text{new}}(q) = B_\epsilon(q) \setminus \left(\bigcup_{q_i \in Q_{\text{obs}}} B_\epsilon(q_i) \right), \quad (5)$$

where $B_\epsilon(x) = \{z \in \mathbb{R}^d : d(z, E(x)) \leq \epsilon\}$.

Theorem 1: Given two new queries q_1, q_2 with

$$d(E(q_2), E(Q_{\text{obs}})) > d(E(q_1), E(Q_{\text{obs}})) \quad (6)$$

and absolute continuity of probability density $p(y)$ over the embedding space of E , then

$$\mathbb{E}[N_{\text{new}}(q_2)] \geq \mathbb{E}[N_{\text{new}}(q_1)], \quad (7)$$

where

$$\mathbb{E}[N_{\text{new}}(q)] = \int_{C_{\text{new}}(q)} p(y) dy. \quad (8)$$

Proof: We first consider that $d(E(q_1), E(Q_{\text{obs}})) \leq \epsilon$. For q_1 , by overlap with historical queries Q

$$C_{\text{new}}(q_1) = B_\epsilon(q_1) \setminus \left(\bigcup_{q_i \in Q_{\text{obs}}} B_\epsilon(q_i) \right), \quad (9)$$

and

$$\begin{aligned} \mathbb{E}[N_{\text{new}}(q_1)] &= \int_{C_{\text{new}}(q_1)} p(y) dy = \int_{B_\epsilon(q_1)} p(y) dy \\ &\quad - \int_{B_\epsilon(q_1) \cap \left(\bigcup_{q_i \in Q_{\text{obs}}} B_\epsilon(q_i) \right)} p(y) dy. \end{aligned} \quad (10)$$

For q_2 , if $d(E(q_2), E(Q_{\text{obs}})) > \epsilon$,

$$\mathbb{E}[N_{\text{new}}(q_2)] = \int_{B_\epsilon(q_2)} p(y) dy, \quad (11)$$

otherwise, $d(E(q_2), E(Q_{\text{obs}})) \leq \epsilon$ and

$$\begin{aligned} \mathbb{E}[N_{\text{new}}(q_2)] &= \int_{B_\epsilon(q_2)} p(y) dy \\ &\quad - \int_{B_\epsilon(q_2) \cap \left(\bigcup_{q_i \in Q_{\text{obs}}} B_\epsilon(q_i) \right)} p(y) dy. \end{aligned}$$

In both cases, since

$$d(E(q_2), E(Q_{\text{obs}})) > d(E(q_1), E(Q_{\text{obs}})),$$

we have

$$B_\epsilon(q_2) \cap \left(\bigcup_{q_i \in Q_{\text{obs}}} B_\epsilon(q_i) \right) \subseteq B_\epsilon(q_1) \cap \left(\bigcup_{q_i \in Q_{\text{obs}}} B_\epsilon(q_i) \right), \quad (12)$$

which implies

$$\int_{B_\epsilon(q_2) \cap C_{\text{new}}} p(y) dy < \int_{B_\epsilon(q_1) \cap C_{\text{new}}} p(y) dy. \quad (13)$$

Therefore,

$$\mathbb{E}[N_{\text{new}}(q_2)] > \mathbb{E}[N_{\text{new}}(q_1)]. \quad (14)$$

For $d(E(q_1), E(Q_{\text{obs}})) > \epsilon$, both queries have

$$C_{\text{new}}(q_1) = B_\epsilon(q_1), \quad (i = 1, 2).$$

Then, by absolute continuity of $p(y)$

$$\mathbb{E}[N_{\text{new}}(q_1)] = \mathbb{E}[N_{\text{new}}(q_2)]. \quad (15)$$

Combining both cases, we have

$$\mathbb{E}[N_{\text{new}}(q_2)] \geq \mathbb{E}[N_{\text{new}}(q_1)]. \quad (16)$$

□

IV. EMBEZZLER

Our approach aims to reconstruct the private RAG knowledge base of the RAG service provider. The main idea is to infer a surrogate embedding model and use it to probe the embedding space, extracting as many chunks as possible with a minimal number of adversarial queries. To clarify the mechanics of our attack, we distinguish between the *semantic space*, which consists of discrete, human-readable natural language structured by linguistic rules, and the *embedding space*, the high-dimensional continuous vector space produced by the RAG system's encoder. To reconstruct the private RAG knowledge base via the embedding space, we must overcome two primary challenges: (1) accurately identify a surrogate embedding model that is similar to the victim one without direct access; (2) with the surrogate model, strategically generate queries to maximize the retrieval of new chunks while minimizing the number of adversarial queries.

Pipeline. To address these challenges, we propose Embezzler. The core technical challenge addressed by Embezzler is bridging the semantic-vector gap: navigating the continuous embedding space to identify unexplored knowledge coordinates and then mapping these back to discrete, meaningful queries in the semantic space to trigger the retrieval of

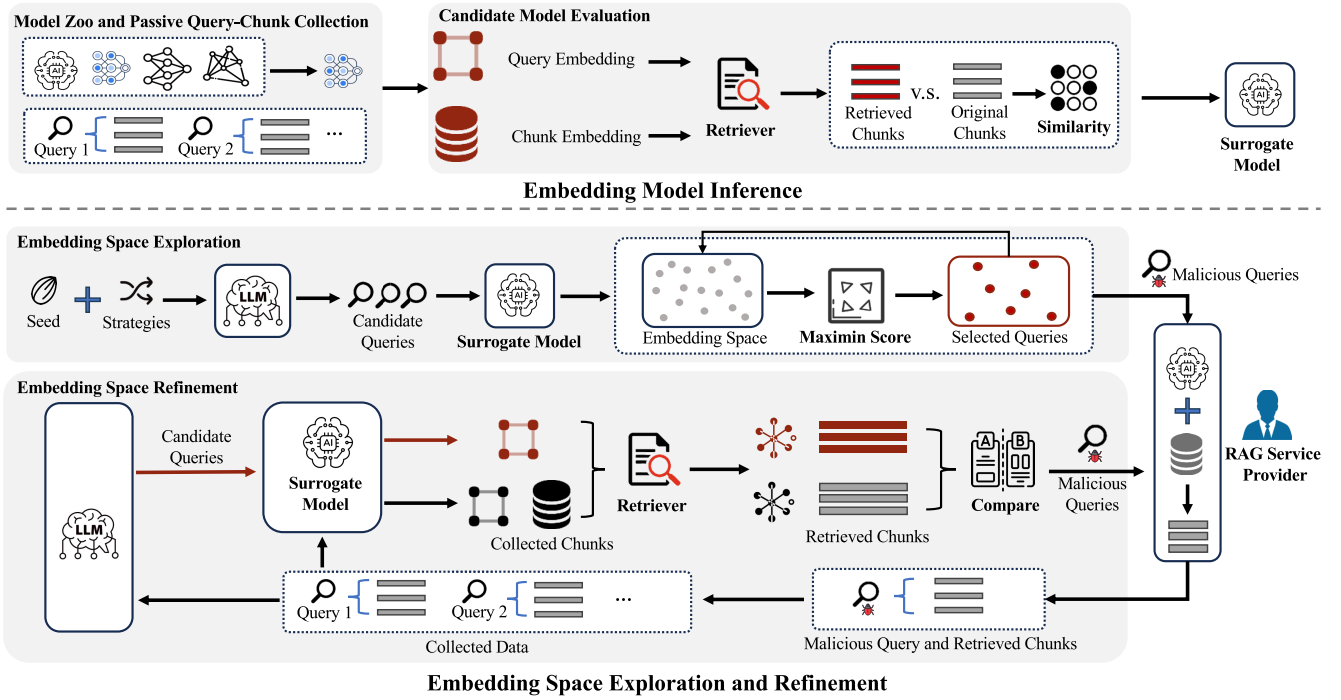


Fig. 3. The workflow of our three-stage attack, Embezzler. We first infer the embedding model from the closed-box RAG service provider. Then, we use the inferred model as a surrogate to systematically exploring and refining the embedding space.

specific private chunks. This method first infers the embedding model of the RAG service provider via the passively observed retrieval behavior without prior knowledge of the architecture or training data of the embedding model. It then efficiently explores and refines the high-dimensional embedding space to expose private chunks. As illustrated in Fig. 3, our approach is of three stages: embedding model inference, embedding space exploration and refinement. For the embedding model inference, we approximate the victim embedding model by selecting a surrogate embedding model using observed query-response pairs. This stage serves as the foundation for achieving both adversarial objectives by overcoming the semantic-vector gap. Then, we aim to build a coarse-grained taxonomy of the knowledge base. By applying a farthest-point sampling strategy in the embedding space, we generate queries that are maximally distant from each other to maximize *Completeness*. The refinement stage focuses on a more fine-grained exploration of the embedding space and ensures *Effectiveness*. Building on the broad overview from the exploration phase, we synthesize new candidate queries by approximating already-retrieved chunks in the embedding space to extract new non-redundant chunks.

Design Rationale. The efficiency of Embezzler relies on the seamless transition from model inference to targeted probing. The surrogate model serves as the adversary's guide within the latent vector space. Without a high-fidelity approximation of the target embedding model, the geometric distances utilized in Stage 2 would fail to accurately reflect underlying semantic relationships. Consequently, establishing an accurate surrogate model is a critical prerequisite to ensure that subsequent exploration avoids redundant queries and achieves high reconstruction efficiency by precisely navigating the target's embedding distribution.

A. Embedding Model Inference

Our core insight is the recognition that different embedding models impose distinct geometric biases on how they map chunks to vectors. This inherent bias fundamentally governs the retrieval behavior. By passively intercepting the query-response pairs of the victim RAG system, we gather indirect evidence of these biases. We first construct a diverse candidate pool of pre-trained embedding models (the model zoo) and collect observed query-response pairs from the RAG service provider. For each candidate model in the zoo, we compute the Jaccard similarity between its generated retrievals and the victim's responses for the same queries. By ranking models based on similarity scores, we select the candidate that most closely aligns with the original retrieval behavior as the surrogate embedding model without requiring the architecture or training data of the victim embedding model.

Model Zoo and Passive Query-Response Collection. We first construct a comprehensive embedding model zoo $\mathcal{M}_{\text{zoo}} = \{E_1, E_2, \dots, E_m\}$ to maximize coverage of potential embedding architectures. Here, each candidate $E_j(\cdot) : \mathbb{T} \rightarrow \mathbb{R}^d$ maps text chunks to a d -dimensional embedding space, where $\mathbb{T}$ is the textual domain. Because different architectures induce distinct geometric biases in $\mathbb{R}^d$, we include models with varying neural architectures. We aggregate publicly available pretrained models spanning dominant Transformer architectures (like MiniLM, MPNet, or DistilBERT) and fine-tuned for specific tasks like semantic search or paraphrase identification. We also passively collect observational data $\mathcal{D}_{\text{obs}} = \{(q_i, R_i) \mid i \in \{1, \dots, n\}\}$, where each $R_i = \text{Ret}(q_i) \subset C$ denotes the top- k retrieved chunks for query q_i of the victim RAG system.

Candidate Model Evaluation. Given the preconstructed model zoo $\mathcal{M}_{\text{zoo}}$ and collected observational data $\mathcal{D}_{\text{obs}}$, we

evaluate the alignment of retrieval behavior between the candidate and victim embedding models. We do this by comparing the top- k chunks retrieved by the victim RAG system with those retrieved by the candidate models for the same set of queries. We quantitatively assess the fidelity of each candidate model and select the most suitable surrogate via the overlap of their retrieval behavior. Specifically, First, for each candidate model E_j , we calculate the embedding vectors for all text chunks C_{obs} in $\mathcal{D}_{obs}$.

Next, we simulate the retrieval behavior of the victim embedding model for each intercepted query-response pair $(q_i, R_i) \in \mathcal{D}_{obs}$. For a query q_i , we encode it using E_j to obtain $\mathbf{q}_{j,i} = E_j(q_i)$, then compute the cosine similarity between $\mathbf{q}_{j,i}$ and all chunk embeddings $\mathbf{c}_{j,l} \in E_j(C_{obs})$

$$s_{j,i}^{(l)} = \frac{\mathbf{q}_{j,i} \cdot \mathbf{c}_{j,l}}{\|\mathbf{q}_{j,i}\| \|\mathbf{c}_{j,l}\|}, \quad \forall l \in \{1, 2, \dots, n\}. \quad (17)$$

The retrieval result of the candidate embedding model E_j , $R'_{j,i}$, is derived by selecting the top- k chunks ranked by $s_{j,i}^{(l)}$

$$R'_{j,i} = \{c_l | s_{j,i}^{(l)} \text{ ranks in the top-}k \text{ of } \{s_{j,i}^{(1)}, \dots, s_{j,i}^{(n)}\}\}. \quad (18)$$

This simulates the retrieval behavior of the victim embedding model as if their system used E_j instead of E .

We then quantify the semantic overlap between the simulated retrieval $R'_{j,i}$ and the observed ground-truth retrieval R_i using the Jaccard similarity metric,

$$Jaccard(R_i, R'_{j,i}) = \frac{|R_i \cap R'_{j,i}|}{|R_i \cup R'_{j,i}|}. \quad (19)$$

Higher values of $Jaccard(R_i, R'_{j,i})$ indicate stronger alignment between E_j and the victim embedding model E . We employ Jaccard similarity because the adversary's objective is the full reconstruction of raw data chunks rather than optimizing downstream generation quality. Since a malicious LLM API provider observes the entire retrieved context window as a collective set, the membership of specific chunks serves as the primary indicator for identifying the underlying embedding model. Thus, set-based overlap provides a more robust and direct signal of model alignment than fine-grained ranking permutations within our specific threat model.

To generalize this across all observed queries, we compute an aggregate similarity score for E_j

$$S_j = \frac{1}{t} \sum_{i=1}^t Jaccard(R_i, R'_{j,i}). \quad (20)$$

The candidate model achieving the highest S_j is selected as the inferred surrogate model

$$E_{\text{surrogate}} = \arg \max_{E_j \in \mathcal{M}_{\text{zoo}}} S_j. \quad (21)$$

B. Embedding Space Exploration

To maximize the efficiency of our reconstruction attack, we leverage the surrogate embedding model to guide our query strategy. The efficiency of the Embezzler pipeline relies on the seamless transition from model inference to targeted probing. The surrogate model obtained in Stage 1 serves as the adversary's guide within the latent vector space; without a

high-fidelity approximation of the target embedding model, the geometric distances utilized in Stage 2 would fail to accurately reflect underlying semantic relationships. Consequently, establishing an accurate surrogate model is a critical prerequisite to ensure that subsequent exploration avoids redundant queries and achieves high reconstruction efficiency by precisely navigating the target's embedding distribution.

We formalize the process of selecting queries for our exploration phase as an adaptive farthest-point sampling problem in the embedding space. We design to build a coarse-grained taxonomy of the knowledge base. We select each new query to maximize its minimum distance from all previously selected queries under the surrogate model. This ensures we prioritize exploring the most under-sampled regions in the embedding space. By systematically expanding our exploration, we maximally map the private knowledge base.

Candidate Query Generation. Formally, given the surrogate model $E_{\text{surrogate}}$ and the set of already retrieved chunks $C_{\text{recon}} \subseteq C$ (initialized with chunks from the observed data $\mathcal{D}_{obs}$), we construct new queries to expand C_{recon} . We first build a coarse-grained taxonomy of the knowledge base by generating a diverse candidate query pool Q_{cand} covering potential semantic variations of the victim knowledge domain. This process ensures our exploration covers the full breadth of the embedding space. We create Q_{cand} by using the observed queries, $Q_{obs} = \{q_1, \dots, q_t\}$, as initial seeds and applying the following strategies:

- **Semantic Augmentation:** To expand the exploration to related topics, we replace key nouns and verbs in the seed queries with synonyms from a resource like WordNet. This allows us to move laterally within the taxonomy, identifying different, but semantically similar, topics (e.g., "economic crisis" $\rightarrow$ "financial recession").
- **Syntactic Variation:** We use rewriting models to generate paraphrases of the seed queries. This strategy explores different grammatical structures and phrasing to identify sub-topics and finer distinctions (e.g., "effects of inflation" $\rightarrow$ "how does rising prices impact markets").
- **Contextual Filling:** By using a masked language model to fill in keywords within our seed queries, we create new, contextually relevant queries. This allows us to discover specific instances or sub-categories within a broader topic, such as generating queries for different types of diseases from a single template (e.g., "treatment for [MASK] disease" $\rightarrow$ "treatment for Alzheimer's disease").

This produces $Q_{\text{cand}} = \{q'_1, \dots, q'_m\}$ with $m \gg t$, ensuring diversity in both syntax and semantics.

Vector-Guided Query Selection. Starting from an empty selected query set $Q_{\text{sel}} = \emptyset$ and reconstructed chunks $C_{\text{recon}} = \emptyset$, we iteratively select queries from Q_{cand} using the surrogate model $E_{\text{surrogate}}$. For each candidate query $q' \in Q_{\text{cand}}$, we compute its minimum distance to existing selected queries

$$d_{\min}(q', Q_{\text{sel}}) = \min_{q \in Q_{\text{sel}}} \left(1 - \frac{E_{\text{surrogate}}(q') \cdot E_{\text{surrogate}}(q)}{\|E_{\text{surrogate}}(q')\| \|E_{\text{surrogate}}(q)\|} \right). \quad (22)$$

We select the query maximizing this minimal distance

$$q^* = \arg \max_{q' \in Q_{\text{cand}}} d_{\min}(q', Q_{\text{sel}}). \quad (23)$$

and update $Q_{\text{sel}} \leftarrow Q_{\text{sel}} \cup \{q^*\}$ and remove q^* from Q_{cand} .

Then, we submit q^* to the victim's system to obtain top- k chunks $R^* = \text{Ret}(q^*)$, then update

$$C_{\text{recon}} \leftarrow C_{\text{recon}} \cup R^* \quad (24)$$

We repeat the above selection process until $|Q_{\text{sel}}|$ reaches a preset budget τ and obtain the reconstructed set C_{recon} .

C. Embedding Space Refinement

The refinement phase is a targeted, fine-grained approach designed to efficiently fill in the gaps of our exploration. After broadly mapping the knowledge base, we leverage the surrogate model to craft adversarial queries that refine the undersampled semantic regions. For each new candidate query, we compute a divergence score based on the Jaccard similarity of its simulated retrievals to those of its nearest existing query. By prioritizing queries with a high divergence score, we efficiently discover new and non-redundant chunks, which ensures a thorough and complete reconstruction of the private knowledge base.

Refinement Query Generation. For the embedding space refinement, we intend to fill in the fine-grained details of the knowledge base. We treat the retrieved chunks as waypoints. From these points, we perturb queries to probe their immediate vector neighborhoods. Given chunks in C_{recon} (those already retrieved), we generate candidate queries by systematically extracting information from the chunks. Specifically, (1) we identify core keywords and concepts to formulate direct, fact-based questions (who, what, when, where) that clarify the fundamental details of a text. (2) We probe for causal and explanatory relationships by generating “why” and “how” questions to understand the motivations, processes, and consequences described. (3) We create questions that extend to related contexts that connects the chunks to broader implications or practical applications to inspire a deeper and more comprehensive exploration. This yields $Q_{\text{syn}}^{(i)} = \{q_{\text{syn}}^{(1)}, \dots, q_{\text{syn}}^{(p)}\}$, where the candidate queries are semantic variation of existing chunks.

Refinement Query Discrimination. To achieve our effectiveness goal, we cannot blindly use these synthetic queries, where many may retrieve redundant chunks. To identify high-value candidates, we leverage $E_{\text{surrogate}}$ as a discriminator to determine whether a query can retrieve new, non-redundant chunks. For each synthetically generated query $q_{\text{syn}}^{(i)}$, we first identify its closest counterpart $q_{\text{nearest}}^{(i)}$ within the existing selected query set Q_{sel} using cosine similarity in the vector space of the surrogate embedding model $E_{\text{surrogate}}$. Next, we simulate the retrieval results $R_{\text{surrogate}}(q_{\text{syn}}^{(i)})$ through $E_{\text{surrogate}}$ while extracting the actual retrieval behavior $R_{\text{nearest}}^{(i)}$ for $q_{\text{nearest}}^{(i)}$.

Then, we compute the divergence score

$$\text{Div}(q_{\text{syn}}^{(i)}) = 1 - \frac{|R_{\text{surrogate}}(q_{\text{syn}}^{(i)}) \cap R_{\text{nearest}}^{(i)}|}{|R_{\text{surrogate}}(q_{\text{syn}}^{(i)}) \cup R_{\text{nearest}}^{(i)}|}, \quad (25)$$

which quantifies the Jaccard divergence between the surrogate-predicted and victim-observed retrievals. This metric guides the selection of queries most likely to probe unexplored regions of the victim embedding space. A high $\text{Div}(\cdot)$ indicates $q_{\text{syn}}^{(i)}$'s surrogate retrievals differ significantly from its closest retrieval behavior of existing query.

We prioritize the query $q_{\text{adv}}^{(i)} = \arg \max \text{Div}(\cdot)$ and submit it to the victim RAG system. The selected query $q_{\text{adv}}^{(i)}$ is submitted to the RAG service provider to retrieve new results R^* , which are added to $C_{\text{retrieved}}$ after deduplication

$$C_{\text{new}} = R^* \setminus C_{\text{retrieved}}. \quad (26)$$

If C_{new} is non-empty, new queries are generated from these chunks

$$Q_{\text{candi}} = Q_{\text{candi}} \cup \text{GenerateQueries}(C_{\text{new}}). \quad (27)$$

To ensure efficient exploration, the process terminates if no new chunks are discovered across β consecutive iterations.

Algorithm 1 Embedding Space Exploration and Refinement

Input: Surrogate model $E_{\text{surrogate}}$, observational data $\mathcal{D}_{\text{obs}}$, budget τ , refinement patience β
Output: Reconstructed chunk set C_{recon}

```

1 // Initialization
2 Initialize  $C_{\text{recon}}, Q_{\text{obs}}$  from  $\mathcal{D}_{\text{obs}}$ ;  $Q_{\text{sel}} \leftarrow \emptyset$ ;
3 // Phase 1: Embedding Space Exploration
4  $Q_{\text{cand}} \leftarrow \text{Generate exploration candidates with } Q_{\text{obs}}$ ;
5 for  $i \leftarrow 1$  to  $\tau$  do
6    $q^* \leftarrow \arg \max_{q' \in Q_{\text{cand}}} d_{\min}(q', Q_{\text{sel}})$ ;
7    $R^* \leftarrow \text{Ret}(q^*)$ ; // Prob victim system
8    $Q_{\text{sel}} \leftarrow Q_{\text{sel}} \cup \{q^*\}$ ;  $Q_{\text{cand}} \leftarrow Q_{\text{cand}} \setminus \{q^*\}$ ;
9    $C_{\text{recon}} \leftarrow C_{\text{recon}} \cup R^*$ ;
10 // Phase 2: Embedding Space Refinement
11  $\text{no\_new\_chunks} \leftarrow 0$ ;
12 while  $\text{no\_new\_chunks} < \beta$  do
13    $Q_{\text{syn}} \leftarrow \text{Generate refinement candidates with } C_{\text{recon}}$ ;
14   foreach  $q_{\text{syn}}^{(i)} \in Q_{\text{syn}}$  do
15     Find its nearest neighbor  $q_{\text{nearest}}^{(i)} \in Q_{\text{sel}}$  using  $E_{\text{surrogate}}$ ;
16     Compute divergence
17      $\text{Div}(q_{\text{syn}}^{(i)}) = 1 - \frac{|R_{\text{surrogate}}(q_{\text{syn}}^{(i)}) \cap \text{Ret}(q_{\text{nearest}}^{(i)})|}{|R_{\text{surrogate}}(q_{\text{syn}}^{(i)}) \cup \text{Ret}(q_{\text{nearest}}^{(i)})|}$ ;
18    $q_{\text{adv}} \leftarrow \arg \max_{q_{\text{syn}} \in Q_{\text{syn}}} \text{Div}(q_{\text{syn}})$ ;
19    $R^* \leftarrow \text{Ret}(q_{\text{adv}})$ ;
20    $C_{\text{new}} \leftarrow R^* \setminus C_{\text{recon}}$ ;
21    $C_{\text{recon}} \leftarrow C_{\text{recon}} \cup C_{\text{new}}$ ;
22   if  $C_{\text{new}} = \emptyset$  then
23      $\text{no\_new\_chunks} \leftarrow \text{no\_new\_chunks} + 1$ ;
24   else  $\text{no\_new\_chunks} \leftarrow 0$ ;
25 // Return the Reconstructed Knowledge Base
26 return  $C_{\text{recon}}$ ;

```

Knowledge Base Reconstruction. The final output of our attack is the reconstructed knowledge base $C_{\text{recon}} = C_{\text{recon}} \cup$

TABLE I
CANDIDATE EMBEDDING MODELS IN THE MODEL ZOO
DURING MODEL INFERENCE

Architecture	Candidate Embedding Models
MPNet	all-mpnet-base-v1
	all-mpnet-base-v2
	multi-qa-mpnet-base-cos-v1
MiniLM	all-MiniLM-L12-v2
	all-MiniLM-L6-v2
	multi-qa-MiniLM-L6-cos-v1
	paraphrase-MiniLM-L3-v2
DistilBERT	all-distilroberta-v1
	multi-qa-distilbert-cos-v1
	msmarco-distilbert-cos-v5
ALBERT	paraphrase-albert-small-v2

C_{new} . This reconstructed knowledge base is a cumulative collection of all unique text chunks retrieved from the victim RAG service provider, where new chunks (C_{new}) in the refinement stage are continuously added. We summarize the embedding space exploration and refinement phases of our Embezzler in Algorithm 1.

V. EXPERIMENTS

A. Evaluation Setups

Datasets. To evaluate the effectiveness of our Embezzler, we utilize three datasets as the target RAG system’s knowledge base. Specifically, We select MS MARCO [37], Natural Questions (NQ) [38], and ChatDoctor [39]. MS MARCO and NQ are well-established, large-scale public datasets for general-purpose document retrieval and question answering. In contrast, ChatDoctor provides a domain-specific challenge with its focus on medical dialogue. For each dataset, we used the first 10,000 consecutive data points as the knowledge base of the target RAG system.

Configurations. In our experiments, the knowledge base of the target RAG system is built from the datasets introduced above. We use the all-mpnet-base-v2 model for embedding generation and employee cosine similarity as the retrieval metric. During each retrieval operation, the system is configured to return the top 5 most similar chunks, i.e., $k = 5$.

The model inference is performed using the candidate models summarized in Table I. These models are all Transformer-based sentence encoders that vary in their core architecture (like MPNet, MiniLM, or DistilBERT) and are fine-tuned for specific tasks like semantic search or paraphrase identification. For the Exploration stage, we generate a large corpus of queries from Wikipedia entries via DeepSeek V3, from which 1,000 queries are randomly sampled. And we select the budget $\tau = 600$. For the Refinement stage, we used DeepSeek V3 to generate subsequent queries from the retrieved document chunks. The refinement is ended if no new chunks are discovered within $\beta = 30$ iterations. We use the prompt in the following box to generate refinement queries with observed chunks.

Baseline. We compare the performance of our method with four SOTA attacks for data reconstruction: LLM-Gen, TGTB, RAG-thief, and Pirates-RAG. LLM-Gen is specifically

designed to perform data reconstruction attacks in the semantic space. TGTB, RAG-thief, and Pirates-RAG are SOTA approaches to reconstruct the knowledge base from the end user. While these methods were not explicitly designed to counter a malicious third-party LLM API provider, they can be modified to achieve the same attack objective.

- **TGTB [20]:** generates queries by randomly sampling from the Common Crawl dataset.
- **RAG-thief [22]:** uses an LLM to expand previously extracted text chunks, both forward and backward, and then uses the expanded content as the new query.
- **Pirates-RAG [21]:** employs a relevance-based mechanism to dynamically track keywords correlated with previously collected chunks and generates a new query using a sample of keywords, which are selected based on their relevance score distribution.
- **LLM-Gen:** In this approach, we use DeepSeek V3 to generate new, related queries for each newly collected chunk. At each iteration, a query is selected from this set to perform the attack, and the process continues until the set is exhausted.

Prompt for Query Generation

You are an expert query generator. Your task is to analyze a given text and generate 3 questions. Derive queries from the following points:

1. Identify Core Keywords and Concepts

Extract the most important nouns, verbs, dates, and proper nouns (names of people, places, or things) from the text. Based on these keywords, generate questions that directly ask about who, what, where, and when. These questions should clarify the fundamental facts presented in the text.

2. Explore Causal and Explanatory Relationships

Look for cause-and-effect relationships, motivations, or processes described in the text. Formulate “why” and “how” questions to probe deeper into the events. Ask about the reasons behind actions, the consequences of events, or the methods by which something occurred.

3. Extend to Related Contexts and Applications

Think about the broader implications of the text. Consider related fields, historical context, or practical applications of the information. Create questions that connect the text’s content to external knowledge. For example, you can ask how the information relates to a similar event, what a hypothetical scenario might look like, or what future developments could be inspired by the text.

Now, please analyze the following text and generate queries based on the points above. Output one query per line.

[Insert your text here]

Metrics. To evaluate our attack methods, we use the Chunk Recovery Rate (CRR) to quantify the number of reconstructed chunks. We also assess the efficiency of data reconstruction using three additional metrics: Chunk Recovery Efficiency (CRE), Query Hit Rate (QHR), and Freshness Rate (FR).

CRR. CRR is calculated as the ratio of the amount of chunks stolen by the attack to the total amount of chunks in the knowledge base of RAG. We define

$$\text{CRR} = \frac{\text{Number of reconstructed chunks}}{\text{Total number of chunks}}. \quad (28)$$

TABLE II
ATTACK EFFICIENCY OF DIFFERENT ATTACK METHODS ON THE THREE DATASETS

		MS MARCO			Natural-questions			Chat-doctor		
		CRE	QHR	FR	CRE	QHR	FR	CRE	QHR	FR
$n=1000$	TGTB	0.517	0.420	0.322	0.606	0.442	0.270	0.639	0.437	0.300
	RAG-thief	0.283	0.260	0.190	0.245	0.229	0.174	0.522	0.440	0.314
	Pirates-RAG	0.594	0.381	0.118	0.565	0.402	0.242	0.553	0.405	0.306
	LLM-Gen	0.221	0.226	0.272	0.289	0.268	0.234	0.472	0.392	0.272
	Embezzler	0.971	0.790	0.564	0.948	0.685	0.406	0.873	0.642	0.426
$n=2000$	TGTB	0.429	0.308	0.190	0.466	0.326	0.194	0.456	0.320	0.192
	RAG-thief	0.208	0.208	0.204	0.183	0.189	0.102	0.459	0.345	0.242
	Pirates-RAG	0.374	0.289	0.106	0.403	0.299	0.180	0.405	0.298	0.104
	LLM-Gen	0.206	0.204	0.160	0.238	0.226	0.196	0.391	0.331	0.222
	Embezzler	0.928	0.592	0.312	0.849	0.530	0.284	0.762	0.515	0.350

A higher CRR value indicates that the attack successfully recovered a larger portion of the knowledge base.

QHR. QHR(n) measures the reconstruction efficiency of an attack within n queries. We define

$$\text{QHR}(n) = \frac{\text{ChunksNum}(n)}{n \times k}, \quad (29)$$

where ChunksNum(n) is the total number of chunks that an attacker has actually obtained within n queries and k is the number of chunks returned in each retrieval. The closer QHR(n) is to 1, the higher the attack efficiency.

CRE. CRE(n) is used to measure the effectiveness of adversarial queries when recovering n chunks. We define

$$\text{CRE}(n) = \frac{n}{k \times \text{QueriesNum}(n)} \quad (30)$$

where QueriesNum(n) is the actual number of queries consumed to obtain n chunks. When QueriesNum(n) is smaller and closer to n/k , the value of CRE(n) is larger and closer to 1, indicating a higher attack efficiency.

FR. FR(n) is to measure the instantaneous efficiency at the n -th query. It is defined as:

$$\text{FR}(n) = \frac{\text{ChunksNumBefor}(n, p)}{p \times k} \quad (31)$$

where ChunksNumBefor(n, p) denotes the total count of reconstructed chunks from the p queries ending at query n . A larger value of FR indicates a higher attack efficiency.

B. Overall Evaluation

Attack Efficiency. We conduct a comparative study to evaluate the attack efficiency of our proposed method against the baselines. On the NQ, MS MARCO, and ChatDoctor datasets, we measure the following metrics: (i) Chunk Recovery Efficiency (CRE) at a target of n recovered chunks, and (ii) the Query Hit Rate (QHR) and Freshness Rate (FR) for the n -th query, with n set to 1,000 and 2,000.

As shown in Table II, our method, Embezzler, consistently outperforms all baselines across all datasets and metrics. For instance, with $n = 1,000$ on the MS MARCO dataset, Embezzler achieves a CRE of 0.971, a QHR of 0.790, and an FR of 0.564. This superior performance is maintained even as the number of queries is increased to $n = 2,000$.

Evolution of Attack Efficiency. We further evaluate the evolution of attack efficiency throughout the attack process on the NQ dataset. The results are presented in Figure 4. Our analysis shows that our Embezzler consistently achieves superior data theft efficiency across all three metrics. As detailed in the subfigures, our Embezzler sustains the highest QHR with a slower rate of decline, maintains a substantially higher CRE as the number of acquired chunks increases, and achieves the best FR for smaller query counts, with the performance gap narrowing as the number of queries grows.

Total Reconstruction Results. To quantify the overall effectiveness of our Embezzler, we measure the CRR values for each attacks. As shown in Figure 5, our method, Embezzler, consistently achieves the highest CRR, with values of 0.89 on MS MARCO, 0.90 on NQ, and 0.89 on ChatDoctor, which indicates that our Embezzler has a superior ability to reconstruct the target knowledge base.

C. Impact of Hyperparameters

1) *Impact of Different Embedding Models:* We now evaluate how the choice of the surrogate embedding model of the malicious LLM API provider influences attack performance. We consider three distinct scenarios:

- **Random:** The attacker does not infer the target model and uses a randomly selected embedding model.
- **Similar:** The attacker successfully infers a model that is semantically similar to the target model.
- **Identical:** The attacker successfully infers the exact model used by the target system.

The second scenario occurs when the attacker's candidate set contains the target model, and the model inference algorithm successfully identifies it. The third scenario, on the other hand, can arise in two situations: (1) the target model is absent from the attacker's candidate set, and the inference algorithm selects the most similar available model; or (2) the target model is present, but the inference algorithm fails to correctly match it, instead choosing a similar model.

We evaluate the attack performance in these three scenarios using three different target embedding models: all-mpnet-base-v2, multi-qa-distilbert-cos-v1, and paraphrase-MiniLM-L3-v2.

The experimental results, presented in Table III, show that attack performance is consistently highest when the attacker and the target use the same embedding model. Furthermore,

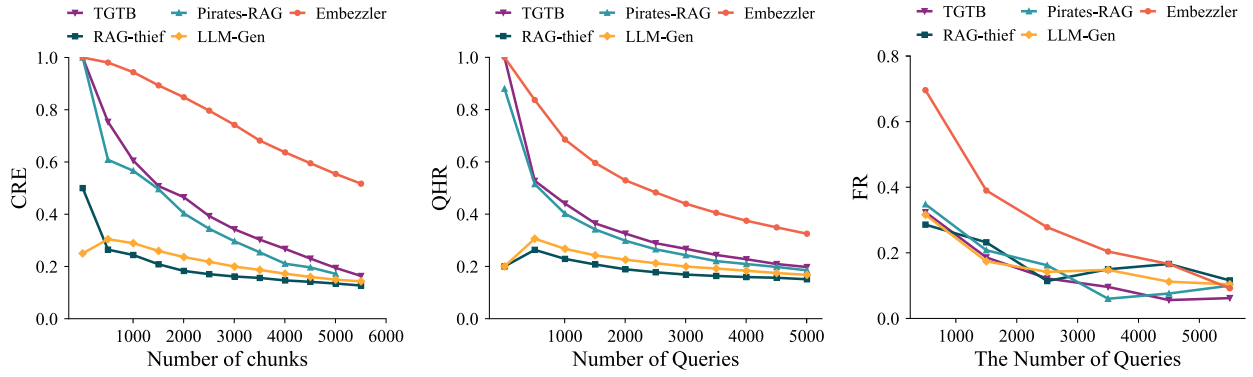


Fig. 4. Evolution of attack efficiency of different attack methods on the NQ dataset.

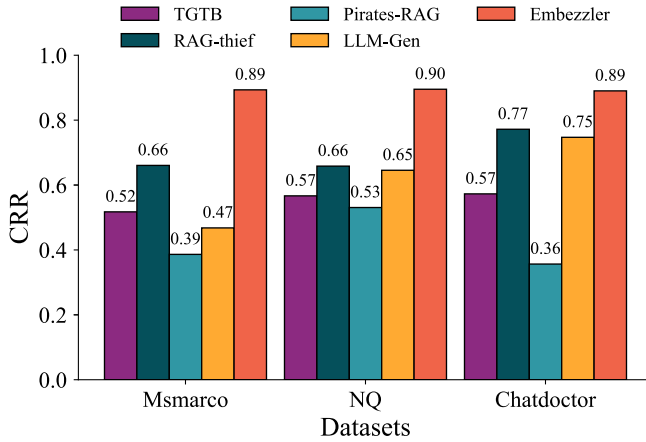


Fig. 5. Total reconstruction results of different attack methods on the Msmarco, NQ, and Chatdoctor datasets.

TABLE III
THE IMPACT OF DIFFERENT EMBEDDING MODELS
ON ATTACK EFFECTIVENESS

Target Model ²	Scenario	CRE	QHR	FR	CRR
all-mpnet-base-v2	Random	0.798	0.508	0.292	0.827
	Similar	0.805	0.527	0.296	0.854
	Identical	0.835	0.532	0.330	0.852
multi-qa-distilbert-cos-v1	Random	0.826	0.524	0.318	0.855
	Similar	0.835	0.542	0.324	0.870
	Identical	0.844	0.549	0.360	0.874
paraphrase-MiniLM-L3-v2	Random	0.795	0.516	0.274	0.829
	Similar	0.816	0.530	0.278	0.831
	Identical	0.872	0.555	0.336	0.855

even when the model inference algorithm identifies only a similar model rather than the exact one, the attack still significantly outperforms the scenario where the attacker uses a randomly selected model.

2) *Impact of the Number of Observed Chunks:* As demonstrated in the previous subsection, the attack is most effective when the attacker successfully infers the same embedding model as the target. A crucial question thus arises: *To what extent can the attacker correctly identify the target's model?* To investigate this, we evaluate the success rate (SR) of our model inference algorithm as a function of the number of chunks available to the attacker. We use two target embedding models for the victim's RAG system: msmarco-distilbert-cos-v5 and all-mpnet-base-v2. The success rate with n chunks

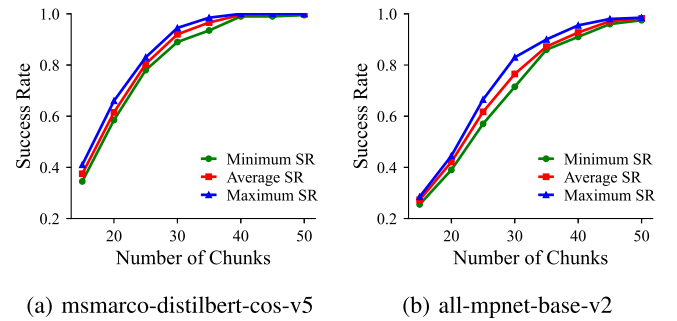


Fig. 6. Success rate (SR) of our embedding model inference under varying numbers of chunks on different target embedding models.

is defined as $SR(n) = r/q$, where q is the total number of inference attempts using different sets of n chunks, and r is the number of successful inferences. The experimental results are presented in Figure 6. To mitigate the impact of random fluctuations, the SR was computed five times, and its maximum (blue line), mean (red line), and minimum (green line) values are recorded.

We have two key observations: (1) our proposed model inference algorithm demonstrates high stability, with minimal variation between maximum and minimum success rates in most cases; (2) the inference success rate increases with the number of chunks, in some cases reaching 100%. Specifically, for the target embedding model msmarco-distilbert-cos-v5, the maximum success rate reaches 83% with only 25 chunks, and increases to 96.6% (average) and 98.5% (maximum) at 35 chunks, and 99.6% (average) and 100% (maximum) at 40 chunks. For the target embedding model all-mpnet-base-v2, 30 chunks are sufficient to achieve an 83% maximum success rate, which further increases to 97.1% (average) and 98% (maximum) at 45 chunks.

3) *Impact of the Number of Exploration Queries:* We now evaluate how the number of queries in the embedding space exploration stage affects attack performance. We present the QHR values at the 2,000-th query and the CRE values at the 5,000-th recovered chunk. The results are summarized in Table IV.

For CRE and QHR, both metrics show an initial increase followed by a gradual decline. Specifically, CRE rises from 0.511 (200 queries) to a peak of 0.568 (800 queries), before decreasing to 0.512 (2,000 queries). QHR follows a similar

TABLE IV
IMPACT OF THE NUMBER OF EXPLORATION QUERIES
ON ATTACK EFFECTIVENESS

# Queries	CRE	QHR	CRR
200	0.511	0.507	0.816
500	0.555	0.529	0.842
800	0.568	0.539	0.853
1000	0.565	0.533	0.859
1200	0.551	0.532	0.852
1500	0.540	0.523	0.858
1700	0.532	0.519	0.860
2000	0.512	0.506	0.858

TABLE V
ATTACK EFFICIENCY COMPARISON OF EMBEZZLER, LLM-GEN, AND
RAG-THIEF UNDER DIFFERENT KNOWLEDGE BASE SIZES

Metrics	Methods	Knowledge Base Size				
		5000	10000	20000	30000	40000
CRE	Embezzler	0.639	0.835	0.926	0.941	0.964
	LLM-Gen	0.159	0.189	0.284	0.314	0.314
	RAG-thief	0.173	0.179	0.266	0.297	0.351
QHR	Embezzler	0.521	0.689	0.806	0.864	0.897
	LLM-Gen	0.180	0.215	0.298	0.327	0.328
	RAG-thief	0.210	0.209	0.285	0.306	0.353

trend, peaking at 0.539 (800 queries) before a slight drop. These results suggest that a moderate number of queries achieves the highest attack efficiency. Insufficient queries limit the attacker's ability to comprehend the topic space, while excessive queries introduce redundancy or noise, thus reducing efficiency.

For CRR, the values remain consistently high, starting at 0.816 for 200 queries and reaching a maximum of 0.860. This stability indicates that even with a relatively small number of queries, the total proportion of recoverable data remains substantial. Consequently, variations in the number of queries primarily affect efficiency-oriented metrics (CRE and QHR) rather than the overall CRR.

4) *Impact of Knowledge Base Size*: To examine how the knowledge base size of a RAG system influences attack effectiveness, we evaluate the attack efficiency and chunk recovery rates of LLM-Gen, RAG-thief, and our proposed method, Embezzler, across different knowledge base sizes. We select LLM-Gen and RAG-thief for this comparison based on their superior overall performance among the baseline methods, as shown in Section V-B. The experiments are conducted on the NQ dataset. For each method, we measured the QHR values at the 1,000-th query and the CRE values at the 2,000-th recovered chunk.

Table V summarizes the attack efficiency comparison of the involved methods under varying knowledge base sizes. Our method, Embezzler, consistently demonstrates superior efficiency, with both CRE and QHR values increasing as the knowledge base size grows. The baseline methods, LLM-Gen and RAG-thief, show only marginal improvements in efficiency, which means that our approach is significantly more scalable and effective in exploring and reconstructing data from large knowledge bases.

Figure 7 illustrates the total reconstruction results, i.e., CRR values. We observe that the graph clearly shows that

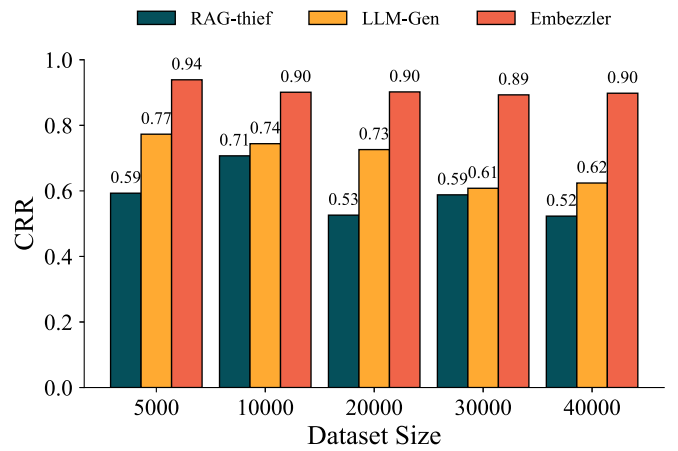


Fig. 7. Total reconstruction results of Embezzler, LLM-Gen, and RAG-thief under different knowledge base sizes.

TABLE VI
COMPONENT-WISE ABLATION

Algorithm	CRE	QHR	FR	CRR
Complete	0.626	0.532	0.330	0.894
w/o Inference	0.605	0.523	0.308	0.866
w/o Probing	0.483	0.453	0.274	0.823
w/o Refinement	0.621	0.506	0.240	0.757

our method, Embezzler, achieves a consistently high CRR of around 0.90 across all tested knowledge base sizes, from 5,000 to 40,000 chunks. This indicates a remarkable ability to recover the vast majority of the knowledge base regardless of its scale. Conversely, the CRR of the baseline methods, LLM-Gen and RAG-thief, shows a negative correlation with knowledge base size, with their total recovery rate decreasing as the knowledge base gets larger. This confirms that our method not only operates more efficiently but also successfully recovers a much larger proportion of the target data, particularly in large-scale scenarios.

D. Ablation Study

To validate the contribution of each component in our Embezzler, we conduct an ablation study by systematically disabling individual components and evaluating the resulting performance. When evaluating the Model Inference component, we replace it with a randomly selected embedding model rather than removing it. This is because the other two stages require a model to operate in vector space.

Table VI compares our complete Embezzler with configurations where individual components are disabled (w/o Inference, w/o Probing, w/o Refinement). The RE is computed from the first 4,000 recovered chunks, while the QHR and FR are derived from the results of the initial 2,000 queries. As demonstrated in the table, the complete Embezzler consistently achieves superior results across all four metrics compared to any ablated variant. This indicates that each component plays a distinct role in enhancing the overall effectiveness.

We also have three observations: (1) Model Inference component enhances the overall effectiveness of the attack, as its absence degrades both the efficiency and total quantity of

chunk reconstruction. (2) Model Space Exploration is essential for boosting efficiency, as its removal leads to the most significant drops in CRE and QHR from 0.626 to 0.483 and from 0.532 to 0.453, respectively. (3) the Content Refinement component plays a critical role in increasing the total amount of data recovered, with its removal causing the most pronounced decrease in the CRR from 0.894 to 0.757. These observations validate that all three stages are indispensable for a successful and efficient data reconstruction attack.

E. Adaptive Defenses

Our Embezzler reveals significant privacy vulnerabilities in RAG systems with LLM API providers. To mitigate our attack, we propose two strategies to fragment or contaminate the data an adversary can observe.

Diversified LLM API Providers. Instead of relying on a single third-party LLM API provider, a RAG service provider can dynamically select from a pool of two or more providers. For instance, the service provider can implement a randomized or round-robin scheduling algorithm to route queries to different providers. With such a strategy, the malicious LLM API provider only observes a fragmented, incomplete stream of queries and their corresponding retrieved chunks. This fragmentation prevents the provider from building a coherent and comprehensive map of the embedding space, which can neutralize the exploration and refinement phrases of our Embezzler. While this strategy enhances security, it introduces system complexity and potential for inconsistent query responses if the LLM API providers exhibit varying performance or behavior.

Data Contamination with a Synthetic Dataset. This approach focuses on actively misleading the malicious provider by contaminating the data they observe. Specifically, the RAG service provider can construct an additional, synthetic dataset that is separate from its sensitive private knowledge base. When a user query is received, the provider strategically intersperses queries related to the fake dataset with the real ones. This can be done according to a pre-defined pattern or at random intervals. The malicious LLM API provider, therefore, observes a mixture of real and fabricated retrieval results. This contamination makes it exceedingly difficult to accurately infer the true embedding space, as the adversary cannot reliably distinguish between real and fake data. However, this defense needs to create a plausible synthetic dataset that is indistinguishable from the real data by the malicious LLM API provider without significantly increasing operational costs.

VI. CONCLUSION

In this paper, we have systematically exposed a previously overlooked privacy vulnerability in Retrieval-Augmented Generation (RAG) systems deployed within agentic communication infrastructures. We demonstrated that malicious third-party LLM API providers can act as powerful adversaries capable of compromising an entire knowledge base. To quantify this threat, we introduced Embezzler, a novel three-stage attack designed to reconstruct sensitive database content from a closed-box RAG system. Our Embezzler first infer

its embedding model and then use this surrogate model to explore and refine the embedding space with minimal queries. Our comprehensive evaluation across three distinct datasets confirms that our Embezzler significantly outperforms existing baselines, achieving a database recovery rate of nearly 90%. As RAG-enabled agents become pervasive in safety-critical domains, our results show the severe privacy risks of deploying RAG systems with third-party LLM APIs and underscore the urgent need for more robust privacy-preserving solutions.

Future Work. While our proposed Embezzler framework achieves high reconstruction completeness with minimal query overhead, several avenues remain for future exploration to further understand and mitigate these systemic risks. First, our current threat model assumes a static knowledge base. However, many real-world agentic AI systems employ dynamic RAG architectures where documents are frequently updated or re-indexed. Future work could investigate adaptive sampling strategies that can detect and capture incremental updates in the embedding space in real-time. Additionally, expanding the attack to multi-modal RAG systems—where retrieval involves images, audio, or structured database entries—presents a significant challenge in bridging cross-modal semantic gaps during the refinement phase.

REFERENCES

- [1] Y. Chang et al., “A survey on evaluation of large language models,” *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 6, pp. 1–39, Oct. 2023.
- [2] X. Bi et al., “Deepseek LLM: Scaling open-source language models with longtermism,” 2024, *arXiv:2401.02954*.
- [3] R. Anil et al., “Gemini: A family of highly capable multimodal models,” 2023, *arXiv:2312.11805*.
- [4] J. Achiam et al., “Gpt-4 technical report,” 2023, *arXiv:2303.08774*.
- [5] W. Fan et al., “A survey on RAG meeting LLMs: Towards retrieval-augmented large language models,” in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 6491–6501.
- [6] P. Zhao et al., “Retrieval-augmented generation for AI-generated content: A survey,” 2024, *arXiv:2402.19473*.
- [7] S. Ma et al., “Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation,” in *Proc. Int. Conf. Learn. Represent.*, 2024, pp. 52782–52806.
- [8] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *Proc. 34th USENIX Security Symp.*, 2025, pp. 3827–3844.
- [9] J. Chen, H. Lin, X. Han, and L. Sun, “Benchmarking large language models in retrieval-augmented generation,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 16, pp. 17754–17762.
- [10] Y. Liu, “PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation,” 2025, *arXiv:2405.02580*.
- [11] N. Maleki, B. Padmanabhan, and K. Dutta, “AI hallucinations: A misnomer worth clarifying,” in *Proc. IEEE Conf. Artif. Intell. (CAI)*, Jun. 2024, pp. 133–138.
- [12] L. Huang et al., “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–55, Mar. 2025.
- [13] W. Xiao et al., “Detecting and mitigating hallucination in large vision language models via fine-grained AI feedback,” in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 24, pp. 25543–25551.
- [14] M. Lazuka, A. Anghel, and T. Parnell, “LLM-pilot: Characterize and optimize performance of your LLM inference services,” in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2024, pp. 1–18.
- [15] X. Hou, J. Han, Y. Zhao, and H. Wang, “Unveiling the landscape of llm deployment in the wild: An empirical study,” 2025, *arXiv:2505.02502*.
- [16] H. Wang, S. Guo, J. He, H. Liu, T. Zhang, and T. Xiang, “Model supply chain poisoning: Backdooring pre-trained models via embedding indistinguishability,” in *Proc. ACM Web Conf.*, Apr. 2025, pp. 840–851.

- [17] H. Huynh, Q.-T. Le, T. N. Nguyen, and V. Nguyen, "Using LLM for mining and testing constraints in API testing," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 2486–2487.
- [18] M. Liu, S. Zhang, and C. Long, "Mask-based membership inference attacks for retrieval-augmented generation," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 2894–2907.
- [19] Y. Li, G. Liu, C. Wang, and Y. Yang, "Generating is believing: Membership inference attacks against retrieval-augmented generation," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2025, pp. 1–5.
- [20] S. Zeng et al., "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in *Proc. Findings Assoc. Comput. Linguistics ACL*, 2024, pp. 4505–4524.
- [21] C. Di Maio, C. Cosci, M. Maggini, V. Poggioni, and S. Melacci, "Pirates of the RAG: Adaptively attacking LLMs to leak knowledge bases," 2024, *arXiv:2412.18295*.
- [22] C. Jiang, X. Pan, G. Hong, C. Bao, Y. Chen, and M. Yang, "Feedback-guided extraction of knowledge base from retrieval-augmented LLM applications," 2024, *arXiv:2411.14110*.
- [23] B. Wang et al., "Unveiling privacy risks in LLM agent memory," 2025, *arXiv:2502.13172*.
- [24] L. de Castro, D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, "EncryptedLLM: Privacy-preserving large language model inference via GPU-accelerated fully homomorphic encryption," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 12677–12688.
- [25] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "BOLT: Privacy-preserving, accurate and efficient inference for transformers," in *Proc. IEEE Symp. Secur. Privacy*, May 2024, pp. 4753–4771.
- [26] M. Guo et al., "Nimbus: Secure and efficient two-party inference for transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 21572–21600.
- [27] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. V. D. Maaten, "CrypTen: Secure multi-party computation meets machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 4961–4973.
- [28] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, Dec. 2020.
- [29] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.
- [30] Y. Ding, C. Niu, F. Wu, S. Tang, C. Lyu, and G. Chen, "Enhancing on-device LLM inference with historical cloud-based LLM interactions," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 597–608.
- [31] M. Anderson, G. Amit, and A. Goldstein, "Is my data in your retrieval database? Membership inference attacks against retrieval augmented generation," in *Proc. 11th Int. Conf. Syst. Secur. Privacy*, 2025, pp. 474–485.
- [32] A. Naseh, Y. Peng, A. Suri, H. Chaudhari, A. Oprea, and A. Houmansadr, "Riddle me this! Stealthy membership inference for retrieval-augmented generation," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2025, pp. 1245–1259.
- [33] Z. Qi, H. Zhang, E. P. Xing, S. M. Kakade, and H. Lakkaraju, "Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems," in *Proc. ICLR Workshop Navigating Addressing Data Problems Found. Models*, 2024, pp. 48733–48755.
- [34] Y. Peng, J. Wang, H. Yu, and A. Houmansadr, "Data extraction attacks in retrieval-augmented generation via backdoors," 2024, *arXiv:2411.01705*.
- [35] S. Cohen, R. Bitton, and B. Nassi, "Unleashing worms and extracting data: Escalating the outcome of attacks against rag-based inference in scale and severity using jailbreaking," *arXiv preprint arXiv:2409.08045*, 2024.
- [36] H. Gurkan and F. de Véricourt, "Contracting, pricing, and data collection under the AI flywheel effect," *Manage. Sci.*, vol. 68, no. 12, pp. 8791–8808, Dec. 2022.
- [37] T. Nguyen et al., "MS MARCO: A human generated machine reading comprehension dataset," 2016, *arXiv:1611.09268*.
- [38] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 453–466, Aug. 2019.
- [39] Y. Li, Z. Li, K. Zhang, R. Dan, S. Jiang, and Y. Zhang, "ChatDoctor: A medical chat model fine-tuned on a large language model meta-AI (LLaMA) using medical domain knowledge," *Cureus*, vol. 15, no. 6, 2023, Art. no. e40895.



Lishuang Hu received the bachelor's and master's degrees from Chongqing University in 2017 and 2020, respectively, where she is currently pursuing the Ph.D. degree with the College of Computer Science. Her research interests include trustworthy machine learning. She is particularly interested in security and privacy threats and defenses in RAG systems.



Hao Wang received the bachelor's degree from Chongqing University in 2022, where he is currently pursuing the Ph.D. degree with the College of Computer Science. His research interests include trustworthy machine learning. He is particularly interested in security threats and defenses in large language model based agents and autonomous systems.



Tao Xiang (Senior Member, IEEE) received the B.Eng., M.S., and Ph.D. degrees in computer science from Chongqing University, China, in 2003, 2005, and 2008, respectively. He is currently a Professor with the College of Computer Science, Chongqing University. He has published over 100 papers in international journals and conferences. His research interests include multimedia security, cloud security, data privacy, and cryptography. He also served as a referee for numerous international journals and conferences.



Zhuo Chen (Member, IEEE) received the Ph.D. degree from Beijing Institute of Technology, Beijing, China, in 2025. He is currently an Assistant Research Scientist with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His current research interests include blockchain technology and covert communication.



Jialing He (Member, IEEE) received the M.S. and Ph.D. degrees from Beijing Institute of Technology, Beijing, China, in 2018 and 2022, respectively. She is currently an Associate Professor with the College of Computer Science, Chongqing University, Chongqing, China. Her current research interests include differential privacy and user behavior mining.



Shangwei Guo (Member, IEEE) received the Ph.D. degree in computer science from Chongqing University, Chongqing, China, in 2017. He was a Post-Doctoral Research Fellow with The Hong Kong Baptist University and Nanyang Technological University from 2018 to 2020. He is currently an Associate Professor with the College of Computer Science, Chongqing University. His research interests include machine learning security, cloud/edge computing security, and database security.